

TRABALHO PARA A DISCIPLINA DE TÉCNICAS DE PROGRAMAÇÃO DO CURSO DE SISTEMAS DE INFORMAÇÃO DA UTFPR: *Dia C++*

Arthur Ferrari Ulbrich, Heitor Garcia Piovan
ulbrich@alunos.utfpr.edu.br, piovan@alunos.utfpr.edu.br

Disciplina: **Técnicas de Programação – CSE20 / S73** – Prof. Dr. Jean M. Simão
Departamento Acadêmico de Informática – DAINF - Campus de Curitiba
Curso Bacharelado em Sistemas de Informação
Universidade Tecnológica Federal do Paraná - UTFPR
Avenida Sete de Setembro, 3165 - Curitiba/PR, Brasil - CEP 80230-901

Resumo – A disciplina de Técnicas de Programação exige o desenvolvimento de um *software* de plataforma, no formato de um jogo, para fins de aprendizado de técnicas de engenharia de *software*, particularmente de programação orientada a objetos em C++. Para tal, neste trabalho, escolheu-se o jogo Dia C++, no qual o jogador enfrenta inimigos em um cenário ambientado na Segunda Guerra Mundial. O jogo tem duas fases que se diferenciam por dificuldades para o jogador. Para o desenvolvimento do jogo foram considerados os requisitos textualmente propostos e elaborado modelagem (análise e projeto) via Diagrama de Classes em Linguagem de Modelagem Unificada (*Unified Modeling Language - UML*) usando como base um diagrama assaz genérico e prévio proposto. Subsequentemente, em linguagem de programação C++, realizou-se o desenvolvimento que contemplou os conceitos usuais de Orientação a Objetos como Classe, Objeto e Relacionamento, bem como alguns conceitos ditos avançados como Classe Abstrata, Polimorfismo, Gabaritos, Persistências de Objetos por Arquivos, Sobre carga de Operadores e Biblioteca Padrão de Gabaritos (*Standard Template Library - STL*). Depois da implementação, os testes e uso do jogo feitos pelos próprios desenvolvedores demonstraram sua funcionalidade conforme os requisitos e a modelagem elaborada. Por fim, salienta-se que o desenvolvimento em questão permitiu cumprir o objetivo de aprendizado visado.

Palavras-chave ou Expressões-chave: Artigo-Relatório para o Trabalho em Técnicas de Programação, Trabalho Acadêmico Voltado a Implementação em C++.

INTRODUÇÃO

O presente projeto é desenvolvido no contexto de uma avaliação parcial da disciplina de Técnicas de Programação, ministrada pelo Prof. Dr. Jean M. Simão, com o objetivo de exercitar e aprimorar técnicas de programação orientada a objetos em C++, bem como consolidar conceitos fundamentais de engenharia de software.

O objeto de estudo e implementação deste trabalho consiste em um jogo de plataformas, previamente acordado com o professor, no qual o jogador deve superar inimigos e obstáculos para avançar nas fases, acumular pontuação e atingir os objetivos propostos. A implementação do jogo segue requisitos específicos e um diagrama de classes em UML fornecido como base, garantindo a aplicação dos conceitos teóricos discutidos em sala de aula.

O método utilizado para o desenvolvimento do projeto baseia-se em um ciclo simplificado de Engenharia de Software, que inclui a compreensão dos requisitos previamente fornecidos, a modelagem e interpretação de diagramas de classes em UML, a implementação de conceitos em C++ com foco na orientação a objetos e a realização de testes para validação do software. Essa abordagem permite a aplicação prática dos conceitos estudados, desde a análise até a execução do projeto.

Este trabalho está organizado em seções que detalham cada etapa da elaboração do projeto. Na sequência, serão apresentados a contextualização e os componentes do jogo, os detalhes do processo de desenvolvimento, os requisitos e conceitos aplicados, bem como as conclusões e as contribuições do projeto para o aprimoramento dos conhecimentos teóricos e práticos abordados na disciplina. Por fim, é discutido como se deu a divisão de tarefas entre os membros da dupla.

EXPLICAÇÃO DO JOGO EM SI

O jogo desenvolvido recebe o nome de "Dia C++", uma clara alusão à famosa operação "Dia D", que ocorreu no final da Segunda Guerra Mundial. A substituição da letra "D" por "C++" faz referência à linguagem de programação utilizada no desenvolvimento do jogo. Com essa escolha, o título sugere uma temática militar, que é refletida nos visuais do jogo, como nas aparências dos personagens e da ambientação.

Dessa forma, o jogo se enquadra no gênero de plataforma 2D, onde um ou dois jogadores devem se locomover, neutralizar inimigos (por meio da capacidade de disparar projéteis) e evitar obstáculos, com o objetivo de alcançar a bandeira vermelha sem que seus pontos de vida se esgotem.

O jogo consiste em duas fases distintas, cada uma com sua própria ambientação, inimigos e obstáculos exclusivos. A quantidade e disposição dos obstáculos e inimigos são geradas aleatoriamente dentro de um limite mínimo e máximo, garantindo variação e desafio ao longo do jogo.

Fase 1: Praia

A primeira fase, chamada "Praia", possui uma temática litorânea, representando o desembarque das forças aliadas na costa francesa. Essa fase contém plataformas e inimigos do tipo "Melee", que, como o nome indica, causam dano corpo a corpo. Esses inimigos possuem uma aparência acinzentada, portam uma arma branca em mãos, perseguem o jogador ao localizá-lo e causam dano ao encostar nele.

Além dos inimigos "Melee", essa fase possui um tipo exclusivo de inimigo chamado "Atirador". Esse inimigo é capaz de disparar projéteis contra os jogadores, causando dano. Além disso, ao colidirem com os jogadores, podem empurrá-los e também causar dano, embora não se movimentem ativamente.

Outro elemento exclusivo da fase "Praia" é o obstáculo "Arame Farpado". Esse obstáculo reduz pontos de vida dos jogadores ao entrarem em contato com eles.

Imagem da Fase 1 (Praia) a seguir:



Figura 1. Fase Praia (primeira fase, com instâncias aleatórias).

Fase 2: Trincheira

A segunda fase, chamada "Trincheira", é ambientada em estruturas que remetem ao nome da fase, típicas em campos de batalha da Segunda Guerra. Assim como a primeira fase, contém plataformas e inimigos do tipo "Melee". No entanto, esta fase apresenta um inimigo exclusivo, o "Tanque". Esse inimigo é um tanque de guerra capaz de disparar projéteis que causam dano aos jogadores. Caso colida diretamente com um jogador, ele o neutraliza instantaneamente. Além disso, possui uma grande quantidade de pontos de vida, tornando-o um inimigo difícil.

Outro elemento exclusivo da fase "Trincheira" é o obstáculo "Fosso". Quando um jogador colide com ele, é automaticamente teleportado para o início da fase, além de ter a possibilidade de sofrer dano.

Imagem da Fase 2 (Trincheira) a seguir:



Figura 2. Fase Trincheira (segunda fase, com instâncias aleatórias).

Menus

Além disso, o jogo possui 4 menus, cada qual com suas opções e funcionalidades.

Menu Principal

O primeiro deles é o Menu Principal, onde o jogador pode escolher entre 6 opções. Quatro destas representam a escolha de fase 1 ou 2, cada uma com a opção de singleplayer (1 jogador) ou multiplayer (2 jogadores). As duas últimas opções são "Carregar", que carrega o último jogo salvo (caso este exista), e "Ranking", que leva o usuário para o menu com o mesmo nome. Imagem desse menu a seguir:

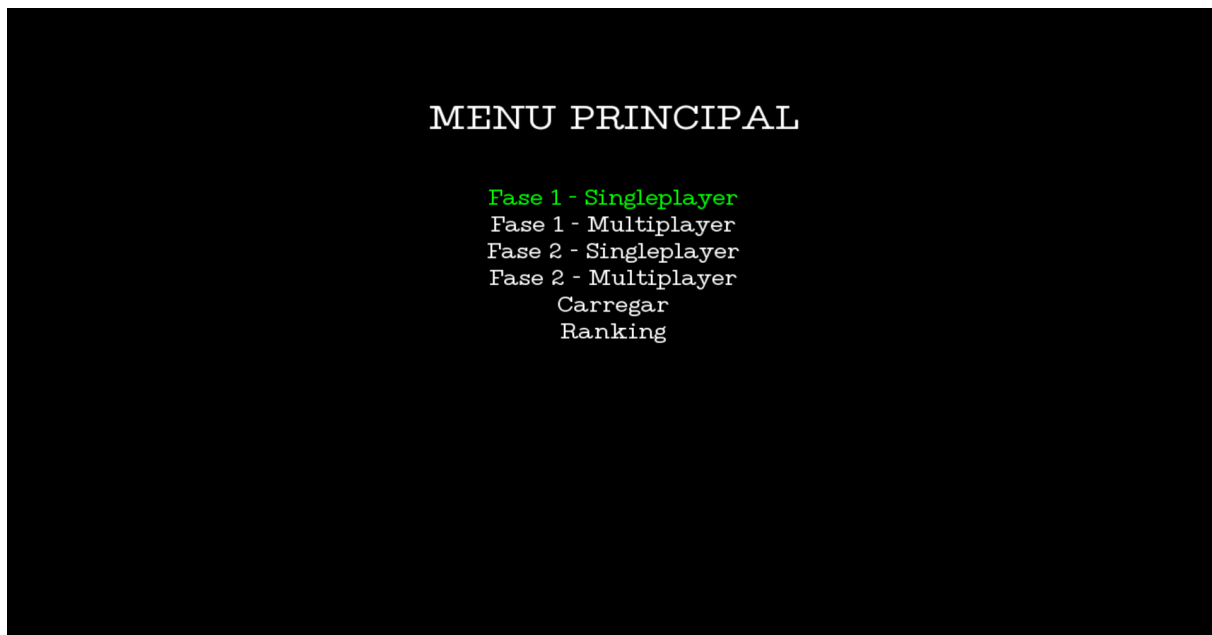


Figura 3. Menu Principal.

Menu de Ranking

No menu de Ranking, é possível checar a classificação de jogadores no formato "Nome - Pontos". Os pontos são obtidos por meio da eliminação de inimigos, sendo distribuídos da seguinte forma: Melee = 5, Atirador = 10, Tanque = 20. Ranking representado na imagem abaixo:



Figura 4. Menu de Ranking.

Menu de Pause

Durante as fases, é possível pausar o jogo, levando ao menu de Pause. Esse menu possui 4 opções: "Continuar" (permite continuar a fase de onde parou), "Salvar" (salva a jogada atual, podendo ser carregada depois), "Cadastrar Nome" (leva ao menu de cadastrar nome, onde o usuário pode escolher seu nome dentro do jogo para salvar pontos nesse cadastro), e "Sair" (que permite ao usuário retornar ao menu principal). Menu de pausa na imagem a seguir:

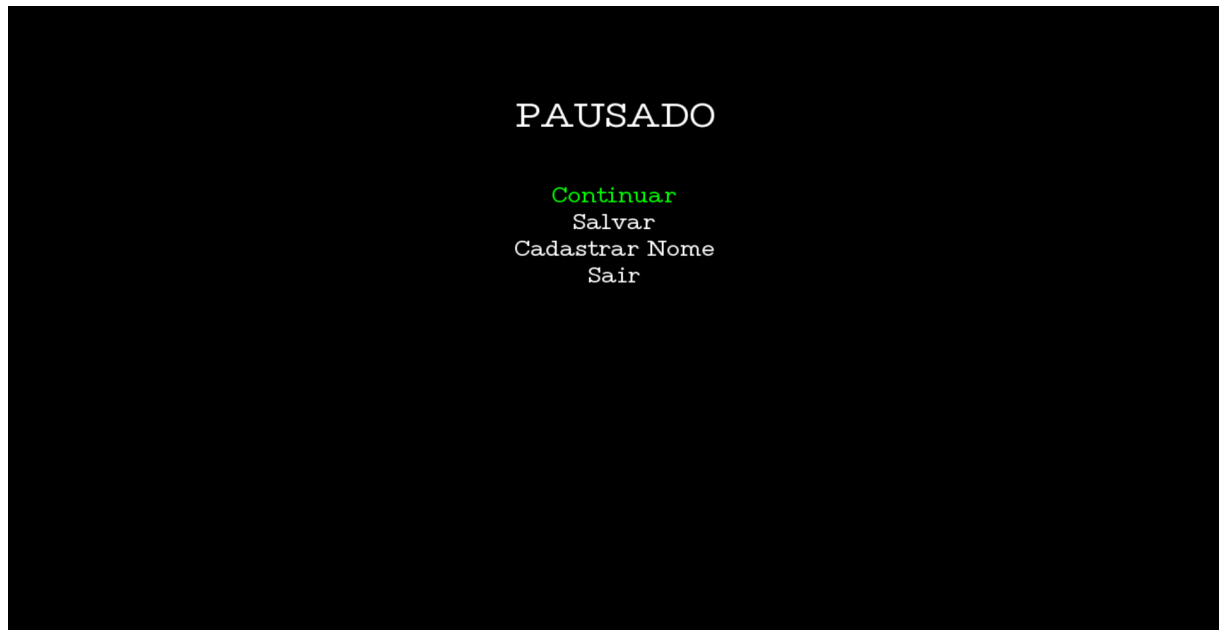


Figura 5. Menu de Pause.

Menu de Cadastro de Nome

O menu de Cadastro de Nome permite que o jogador escolha um nome dentro do jogo, garantindo que seus pontos sejam salvos corretamente no ranking. Menu de cadastro de nome abaixo:

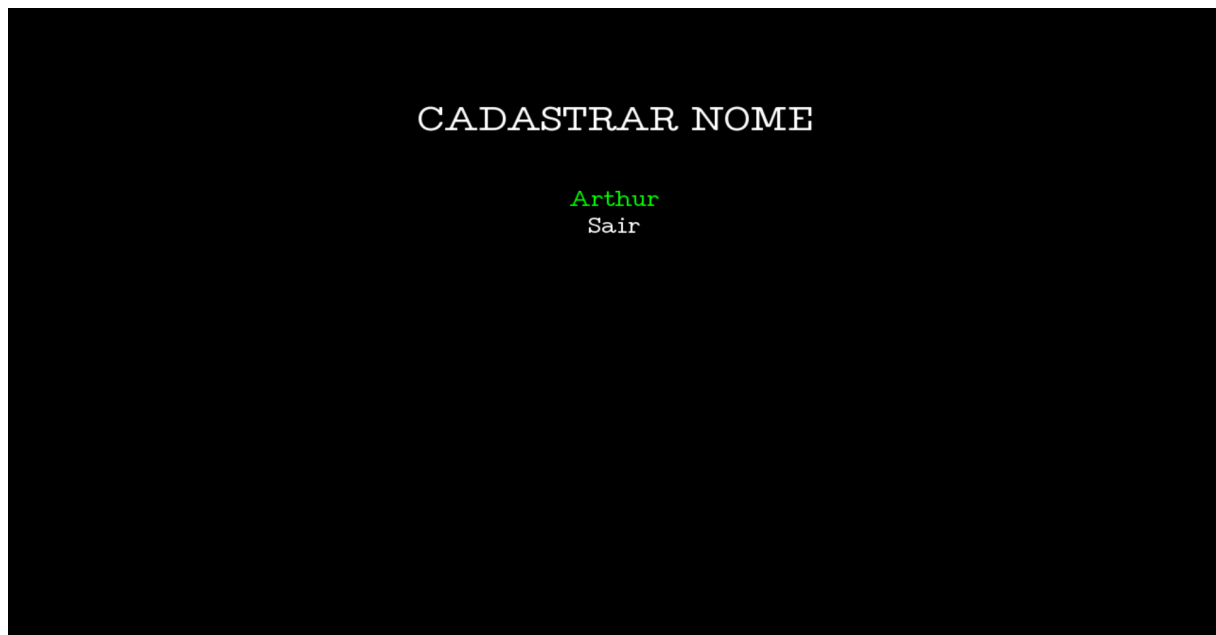


Figura 6. Menu de Cadastro de Nome.

Mapeamento de teclas

Por fim, é relevante destacar o mapeamento de teclas necessárias ao jogar:

- Enter: Confirmar seleção nos menus;
- Up e Down: Alternar entre opções nos menus;
- Esc: Pausar o jogo;
- W, A, S e D: Movimentação JOGADOR 1;
- F: Disparar projéteis JOGADOR 1
- Up, Down, Left e Right : Movimentação do JOGADOR 2;
- M: Disparar projéteis JOGADOR 2.

DESENVOLVIMENTO DO JOGO NA VERSÃO ORIENTADA A OBJETOS

O desenvolvimento do jogo “Dia C++” foi realizado utilizando o paradigma de Orientação a Objetos (OO), o que proporcionou uma estrutura modular e organizada. A utilização desse paradigma permitiu a reutilização de código e a criação de um sistema flexível e de fácil manutenção.

O processo de desenvolvimento seguiu os requisitos previamente estabelecidos na **Tabela 1** (presente abaixo), que orientaram as etapas de modelagem e implementação. A modelagem do sistema foi realizada utilizando diagramas de classes UML, que ajudaram a estruturar as entidades do sistema e suas interações. Com base nessa modelagem, a implementação foi conduzida em C++ utilizando a biblioteca SFML, que forneceu os recursos necessários para a construção da interface gráfica e das interações do jogo/software.

A orientação a objetos foi essencial para garantir uma estrutura organizada e escalável, permitindo que o sistema fosse desenvolvido de maneira eficiente. A utilização do C++ e da SFML possibilitou a criação de um ambiente gráfico interativo, atendendo às necessidades do projeto.

TABELA DE REQUISITOS FUNCIONAIS

A tabela abaixo apresenta os requisitos funcionais previamente definidos para o desenvolvimento do jogo. Ela detalha o status de cada requisito (Realizado ou Não Realizado), a área em que foi implementado e, ao final, apresenta a porcentagem de requisitos atendidos.

Tabela 1. Lista de Requisitos do Jogo e exemplos de Situações.

N.	Requisitos Funcionais	Situação	Implementação
1	Apresentar graficamente menu de opções aos usuários do Jogo, no qual pode se escolher fases, escolher ver colocação (<i>ranking</i>) de jogadores e escolher demais opções pertinentes (previstas nos demais requisitos).	REALIZADO	Classe Menu e seu respectivo objeto, com suporte da SFML.
2	Permitir um ou dois jogadores com representação gráfica aos usuários do Jogo, sendo que no último caso é para que os dois joguem de maneira concomitante.	REALIZADO	Classe Jogador cujos objetos são agregados em jogo.
3	Disponibilizar ao menos duas fases <u>distintas</u> que podem ser jogadas sequencialmente ou selecionadas, via menu, nas quais jogadores tentam neutralizar inimigos por meio de algum artifício e vice-versa.	REALIZADO	Classes Fase, Praia, e Trincheira, no pacote Fases, classe Menu, classes do namespace Inimigos e classe Jogador no pacote Personagens
4	Ter pelo menos três tipos distintos de inimigos, cada qual com sua representação gráfica, sendo que ao menos um deles deve poder lançar projéteis contra o(s) jogador(es) e um dos inimigos deve ser um 'chefão'.	REALIZADO	Pacote Inimigos, Classes Melee, Atirador e Tanque, sendo Atirador e Tanque lançadores de Projéteis.
5	Ter a cada fase ao menos dois tipos de inimigos (<u>um deles exclusivo nela</u>) com número aleatório de instâncias, podendo ser várias instâncias (<u>definindo um máximo</u>) e sendo pelo menos 3 instâncias <u>para cada tipo que estiver na fase</u> .	REALIZADO	Classes: Fase, Praia, Trincheira do pacote Fases.
6	Ter três tipos de obstáculos, cada qual com sua representação gráfica, sendo que ao menos um causa dano em jogador se colidirem.	REALIZADO	Pacote Obstáculos, classes Plataforma. Arame Farpado (causa dano ao colidir) e Fosso
7	Ter em cada fase ao menos dois tipos de obstáculos (<u>um deles exclusivo nela</u>) com número aleatório (<u>definindo um máximo</u>) de instâncias	REALIZADO	Classes: Fase, Praia, Trincheira do pacote Fases.

	(i.e., objetos), sendo pelo menos 3 instâncias por tipo.		
8	Ter em cada fase um cenário de jogo constituído por obstáculos, sendo que parte deles devem ser plataformas ou similares, sobre as quais pode haver inimigos e podem subir jogadores. Em cada fase, só poder ter um tipo coincidente de inimigo e um tipo coincidente de obstáculo (que é a plataforma) em relação às demais fases.	REALIZADO	Pacote Obstáculos, classe Plataforma (obstáculo coincidente) e pacote Personagens, classe Melee (Inimigo coincidente), Pacote Fases: classes Fase, Praia e Trincheira.
9	Gerenciar colisões entre jogador para com inimigos e seus projéteis, bem como entre jogador para com obstáculos. Ainda, todos eles devem sofrer o efeito de alguma 'gravidade' no âmbito deste jogo de plataforma vertical e 2D.	REALIZADO	Classe Gerenciador de Colisões e seu respectivo objeto, com suporte da SFML, gravidade atualizada nas próprias entidades
10	Permitir: (1) salvar nome do usuário, manter/salvar pontuação (incrementada via neutralização de inimigos) do jogador controlado pelo usuário e gerar lista de pontuação (<i>ranking</i>). E (2) Pausar e Salvar/Recuperar Jogada.	REALIZADO	Classes Menu, Eventos, Dia e Classes do namespace Entidades.
Total de requisitos funcionais apropriadamente realizados.			100%

MODELAGEM E ARQUITETURA

A modelagem foi realizada utilizando UML (Unified Modeling Language), linguagem visual para projetar e documentar sistemas de software, tendo como base um diagrama de classes pré-definido pelo professor e expandindo-o conforme as necessidades do projeto, além de adequá-lo conforme ele se desenvolve. As classes constituem os elementos relacionados ao jogo como gerenciadores, listas, entidades, menu, fases, etc. A estrutura do jogo foi dividida em pacotes principais:

- Fases: Contém as classes Fase, Praia e Trincheira, representando os diferentes cenários do jogo.
- Entidades: Inclui a classe projétil e os pacotes Personagens e Obstáculos.
- Personagens: Inclui a classe Jogador e o pacote Inimigos.
- Inimigos: Contém as classes Inimigo (abstrata), Melee, Atirador e Tanque
- Obstáculos: Compreende os elementos interativos do jogo, como Plataforma, Arame Farpado e Fosso.
- Gerenciadores: Contém classes responsáveis pelo controle do jogo, como Gerenciador de Colisões, Gerenciador de Eventos e Gerenciador de Gráfico.
- Listas: Contém classes relacionadas a manipulação de elementos por meio de listas.

A herança foi amplamente utilizada para abstrair comportamentos comuns entre as entidades do jogo, utilizando classes base como Entidade e Personagem. O polimorfismo permitiu implementar métodos genéricos em classes abstratas para manipulação de entidades, enquanto a composição, agregação e associação foram empregadas para estruturar as relações

[illegible]

Tendo em vista o diagrama de classes acima, existem algumas classes que se destacam pela importância no funcionamento do jogo em si. Destas é possível citar primeiramente a classe Dia, que representa a classe principal, onde o jogo é propriamente executado. Outras classes essenciais são Praia e Trincheira, que herdam uma classe fundamental para o jogo, a classe Fase, nela o cenário é criado, salvo e carregado, além de fazer com que todas as entidades executem por meio de Lista Entidades, a qual Fase agrega fortemente. Além dessas, as classes gerenciadores desempenham um papel crucial no programa, eles tornam possível ler inputs, fornecer renderização gráfica e gerenciar colisões entre todas as entidades presentes no jogo.

IMPLEMENTAÇÃO EM C++ E SFML

A implementação utilizou a biblioteca SFML para criar e gerenciar diversas funcionalidades essenciais do projeto. Algumas das principais funcionalidades implementadas utilizando a SFML incluem:

- Leitura de teclas e eventos: O GerenciadorEventos foi responsável por processar a entrada do usuário, garantindo uma jogabilidade responsiva. O sistema capturou eventos como movimentação, ataques e interações com menus.
- Renderização gráfica: Implementada na classe GerenciadorGrafico, que gerencia a exibição de elementos visuais na tela, garantindo um desempenho otimizado e atualização fluida dos quadros.
- Tratamento de colisões: A classe GerenciadorColisoes verificou as interações entre jogadores, inimigos, obstáculos e projéteis, assegurando uma interação consistente entre os elementos do jogo.

Além do uso da SFML, outras técnicas e ferramentas foram empregadas para otimizar a implementação do jogo. A persistência de dados foi implementada utilizando arquivo JSON e arquivo de texto para salvar o progresso do jogador e o ranking de pontuações. Além disso, gabaritos/templates foram utilizados para listas dinâmicas, facilitando a manipulação das entidades dentro do jogo.

O uso extensivo de ponteiros e alocação dinâmica de memória garantiu eficiência no gerenciamento de recursos, enquanto o encapsulamento e a modularização do código facilitaram sua manutenção e expansão.

Além de conceitos de computação, foram utilizados conceitos interdisciplinares de áreas como matemática e física, como por exemplo estatística (probabilidade) e mecânica (aceleração e gravidade).

Foram desenvolvidas algumas leves sofisticações, como no comportamento de inimigos, que localizam e atacam o jogador somente se ele estiver em seu campo de visão (coordenadas ordenadas iguais), fazendo com que os tanques e atiradores disparem projéteis e os melees persigam o jogador.

TABELA DE CONCEITOS UTILIZADOS E NÃO UTILIZADOS

A tabela apresentada a seguir dispõe os conceitos abordados ao longo da disciplina, especificando se foram aplicados no desenvolvimento do projeto. Além disso, são indicados o local de implementação e a maneira como cada conceito foi empregado, constando também o porquê foi usado, proporcionando uma visão detalhada da utilização dos conteúdos estudados na prática do projeto.

Tabela 2. Lista de Conceitos Utilizados e Não Utilizados no Trabalho.

N.	Conceitos	Uso	O quê / Onde & Justificativa em uma frase
1	Elementares:		
1.1 &	- Classes, objetos. & - Atributos (privados), variáveis e constantes. - Métodos (com e sem retorno).	Sim	- Todos .hpp e .cpp, como nas classes nos <i>namespaces</i> Fases e Entidades. - Classes, Objetos, Atributos e Métodos foram utilizados porque são conceitos elementares na orientação a objetos.

1.2 &	- Métodos (com retorno <i>const</i> e parâmetro <i>const</i>). - Construtores (sem/com parâmetros) e destrutores	Sim	- Na maioria dos .hpp e .cpp, como nas classes nos <i>namespaces</i> Personagens e Fases. - A constância pertinente evita mudanças equivocadas, construtores são mandatórios para inicializar atributos e destrutores pertinentes para finalizações como desalocações.
1.3	- Classe Principal.	Sim	- Main.cpp & Principal.hpp/.cpp - Uma classe Principal é mais 'purista' em termos de OO.
1.4	- Divisão em .h e .cpp.	Sim	- No desenvolvimento como um todo, como nas classes nos <i>namespaces</i> Personagens e Fases.. - Permite organizar as classes e afins que compõem o sistema.
2	Relações de:		
2.1	- Associação direcional. & - Associação bidirecional.	Sim	- Em vários dos .hpp e .cpp, como nas classes nos <i>namespaces</i> Gerenciadores e Personagens. - Associação direcional é fundamental para organizar e esclarecer dependências entre classes. - Associação bidirecional é fundamental para representar relações mútuas entre classes, onde ambas dependem uma da outra.
2.2 &	- Agregação via associação. - Agregação propriamente dita.	Sim	- Composição: vários .hpp e .cpp, como nas classes Dia, Fase, Praia e Trincheira. O objeto faz parte de outro, mas ainda pode existir por conta própria, o que ajuda a manter tudo mais organizado e reutilizável. - Agregação fraca: Na classe Fase. Um objeto se refere a outro, mas não controla sua existência, dando mais flexibilidade e permitindo que ele seja usado em outros lugares.
2.3 &	- Herança elementar. - Herança em vários níveis.	Sim	- Elementar: Em alguns dos .hpp e .cpp, como nas classes no namespace Fases. - Vários Níveis: Em alguns dos .hpp e .cpp, como nas classes no namespace Entidades. - Reutilização de código.
2.4	- Herança múltipla.	Não	Conceito não utilizado.
3	Ponteiros, generalizações e exceções		
3.1	- Operador <i>this</i> para fins de relacionamento bidirecional	Sim	- Em dia.cpp e menu.cpp - Usar o <i>this</i> permite passar o próprio objeto como referência para o construtor de outro, criando uma ligação direta entre eles.
3.2	- Alocação de memória (<i>new</i> & <i>delete</i>).	Sim	- Em alguns .cpp como nas classes Jogador, Atirador e Tanque. - <i>new</i> e <i>delete</i> permitem alocação dinâmica e controle manual de memória em C++.
3.3	- Gabaritos/ <i>Templates</i> criados/adaptados pelos autores para Listas.	Sim	- Classes Lista (adaptada), Iterator(criada) e IteratorLista(criada). - Gabaritos/templates adaptados para listas facilitam a criação e organização de estruturas de dados de forma eficiente e reutilizável.
3.4	- Uso de Tratamento de Exceções (<i>try catch</i>).	Sim	- No arquivo Dia.cpp. - O uso de <i>try catch</i> permite capturar e tratar erros de forma controlada, garantindo a continuidade do programa e evitando falhas inesperadas.
4	Sobrecarga de:		
4.1	- Construtoras e Métodos.	Sim	- Em Fase e Jogador (Construtoras), Em Fase (Método) - A sobrecarga de construtores e métodos permite criar múltiplas versões com diferentes parâmetros, oferecendo flexibilidade, reutilização e legibilidade no código.
4.2	- Operadores (2 tipos de operadores pelo menos).	Sim	- Foi usado o operador--() na classe Personagem para decrementar a vida (--), e o operador() na estrutura ComparadorJogador, na classe Jogador, para comparar pontuação entre ponteiros de jogadores. - A sobrecarga de operadores permite personalizar operadores para tipos definidos pelo usuário, tornando o código mais intuitivo.
---	Persistência de Objetos (via arquivo de texto ou binário)		

4.3	- Persistência de Objetos.	Sim	-Em Fase e namespace Entidades, via JSON. -Salvamento de Entidades para manter a jogada.
4.4	- Persistência de Relacionamento de Objetos.	Não	Conceito não utilizado.
5	Virtualidade:		
5.1	- Métodos Virtuais Usuais.	Sim	-Em algumas classes, como Personagem e Entidade. -Métodos virtuais permitem polimorfismo, com comportamentos específicos em subclasses.
5.2	- Polimorfismo.	Sim	-Em várias classes, como Personagem e Entidade -Polimorfismo permite que um objeto se comporte de maneiras diferentes dependendo do contexto.
5.3	- Métodos Virtuais Puros / Classes Abstratas.	Sim	-Em várias classes, como Ente e Fase -Métodos virtuais puros e classes abstratas garantem implementação obrigatória em subclasses, promovendo flexibilidade.
5.4	- Coesão/Desacoplamento efetiva e intensa com o apoio de padrões de projeto (mais de 5 padrões).	Não	Foram utilizados apenas 2 padrões de projeto.
6	Organizadores e Estáticos		
6.1	- Espaço de Nomes (<i>Namespace</i>) criado pelos autores.	Sim	-Namespace Inimigos -Espaços de nomes organizam o código, evitando conflitos de nomes e melhorando a legibilidade e manutenção.
6.2	- Classes aninhadas (<i>Nested</i>) criada pelos autores.	Sim	-Classe IteratorLista aninhado com Iterator no namespace Listas. -Proporcionam melhor encapsulamento e organização, além de contribuir para a legibilidade e a manutenção.
6.3	- Atributos estáticos e métodos estáticos.	Sim	-Em algumas classes como Ente, Jogador e classes do pacote Gerenciadores. -São compartilhados com todas as instâncias de classe.
6.4	- Uso extensivo de constante (<i>const</i>) parâmetro, retorno, método...	Sim	-Na maioria dos .hpp e .cpp. -Evitar erros e alterações indesejadas, promovendo segurança.
7	Standard Template Library (STL) e String OO		
7.1	- A classe Pré-definida <i>String</i> ou equivalente. & <i>Vector</i> e/ou <i>List</i> da STL (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Sim	-Na classe Jogador (vector e string). - String : facilita a manipulação de sequências de caracteres. - Vector : oferecem armazenamento dinâmico e gerenciamento automático de memória para objetos.
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Sim	-Classe Gerenciador Gráfico e Classe Dia. -Mapa: Permite armazenar e acessar pares chave-valor de forma eficiente e ordenada. -Multi-Conjunto: Permite armazenar de forma ordenada elementos, podendo haver repetição de “chave”
---	Programação concorrente		
7.3	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não	NÃO realizado no tempo previsto
7.4	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não	NÃO realizado no tempo previsto
8	Biblioteca Gráfica / Visual		

8.1 &	- Funcionalidades Elementares. - Funcionalidades Avançadas como: tratamento de colisões, duplo <i>buffer</i> ou outros.	Sim	-Tratamento de Colisões (em Gerenciador de Colisões). -Leitura de Eventos (em Gerenciador de Eventos). -Renderização de janela e desenhar elementos em tela.
8.2 O U	- Programação orientada e evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto <i>Observer</i>) em algum ambiente gráfico. - <i>RAD – Rapid Application Development</i> (Objetos gráficos como formulários, botões etc).	Não	Gerenciador de Eventos SEM Observer
---	Interdisciplinaridades via utilização de Conceitos de Matemática Contínua e/ou Física.		
8.3	- Ensino Médio Efetivamente.	Sim	-Gravidade com aceleração (movimento uniformemente acelerado). -Utilizado para fazer com que o jogo tenha gravidade funcional na jogabilidade.
8.4	- Ensino Superior Efetivamente.	Não	-Nenhum conceito utilizado.
9	Engenharia de Software		
9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	Sim	-Os requisitos foram cumpridos por meio da implementação conforme especificações, validação por testes e rastreabilidade dos requisitos. -Garantir compreensão, melhoria e rastreabilidade dos requisitos assegura que o produto atenda às necessidades e facilita a verificação de seu cumprimento.
9.2	- Diagrama de Classes em <i>UML</i> .	Sim	-Respectivo arquivo do Diagrama de Classes -O Diagrama de Classes em UML representa a estrutura do sistema, facilitando o entendimento e a comunicação entre a equipe.
9.3	- Uso efetivo e intensivo de padrões de projeto <i>GOF</i> , <i>i.e.</i> , mais de 5 padrões.	Não	Foram utilizados apenas 2 padrões de projeto.
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Sim	-Garante que o sistema atenda aos requisitos especificados e que as interações entre as classes sejam corretamente implementadas e validadas.
10	Execução de Projeto		
10.1 &	- Controle de versão de modelos e códigos automatizado (via github). - Uso de alguma forma de cópia de segurança (<i>i.e.</i> , <i>backup</i>).	Sim	-Google Drive para backup https://github.com/HeitorGPiovan/Jogo
10.2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	-2 reuniões, a primeira no dia 11/12/2024 e a segunda no dia 18/12/2024, ambas no horário de 15:30 -Emails enviados nos dias acima para o Professor, constando as informações requisitadas. - Garantiu que o projeto estivesse no caminho correto, lapidando e aperfeiçoando o desenvolvimento.

10.3	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	-5 reuniões do PETECO em que ambos os integrantes da dupla participaram -Datas e horários: 1- 29/11/2024, 13:00 até 14:40 2- 05/12/2024, 15:50 até 16:40 3- 06/12/2024, 13:00 até 14:40 4- 12/12/2024, 17:00 até 18:20 5- 13/12/2024, 13:00 até 14:40 - Essencial para se adequar com a biblioteca gráfica, forma de projeto e cumprimento de requisitos.
10.4 &	- Escrita do trabalho e feita da apresentação - Revisão do trabalho escrito de outra equipe e vice-versa.	Sim	- Ambos contribuíram significativamente tanto na escrita do trabalho quanto na elaboração da apresentação de uma maneira geral. -Dupla responsável pela revisão do trabalho: Lucas Schaefer e Matheus Valente
Total de conceitos apropriadamente utilizados.			80%

DISCUSSÃO E CONCLUSÕES

O desenvolvimento deste projeto proporcionou uma experiência prática valiosa, integrando teoria e aplicação. Durante o processo houve dificuldades técnicas, como a implementação de mecânicas e a otimização de recursos, mas esses desafios contribuíram significativamente para o aprendizado. Além de reforçar os conceitos abordados na disciplina, o projeto também permitiu o desenvolvimento de conhecimentos em Engenharia de Software, cumprimento de requisitos, programação e uso de bibliotecas gráficas, entre outros.

Os resultados alcançados foram positivos, com o jogo alcançando seus objetivos principais, cumprindo os requisitos solicitados e demonstrando os conceitos aprendidos em sala de aula. O feedback dos colegas e professores durante o desenvolvimento foi essencial, ajudando a identificar pontos de melhoria e auxiliando na clareza das instruções.

DIVISÃO DO TRABALHO

A tabela a seguir representa a distribuição da responsabilidade atribuída a cada integrante no cumprimento dos requisitos conceituais já mencionados na Tabela 2, evidenciando quem trabalhou/desenvolveu mais cada tópico no projeto, e ao final, logo após a tabela, consta a porcentagem de participação de cada integrante da dupla.

Tabela 3. Lista de Atividades e Responsáveis.

N.	Atividades	Responsáveis
1	Elementares: AMBOS	
1.1	- Classes, objetos. & - Atributos (privados), variáveis e constantes. & - Métodos (com e sem retorno).	Arthur e Heitor
1.2	- Métodos (com retorno <i>const</i> e parâmetro <i>const</i>). & - Construtores (sem/com parâmetros) e destrutores	Arthur e Heitor
1.3	- Classe Principal.	Arthur e Heitor
1.4	- Divisão em .h e .cpp.	Arthur e Heitor

2	Relações de: AMBOS	
2.1	- Associação direcional. & - Associação bidirecional.	Arthur e Heitor
2.2	- Agregação via associação. & - Agregação propriamente dita.	Arthur e Heitor
2.3	- Herança elementar. & - Herança em vários níveis.	Arthur e Heitor
2.4	- Herança múltipla.	Não utilizado
3	Ponteiros, generalizações e exceções: AMBOS	
3.1	- Operador <i>this</i> para fins de relacionamento bidirecional.	Arthur e Heitor
3.2	- Alocação de memória (<i>new & delete</i>).	Arthur e Heitor
3.3	- Gabaritos/ <i>Templates</i> criada/adaptados pelos autores para Listas.	Arthur e Heitor
3.4	- Uso de Tratamento de Exceções (<i>try catch</i>).	Arthur e Heitor
4	Sobrecarga de: AMBOS	
4.1	- Construtoras e Métodos.	Arthur e Heitor
4.2	- Operadores (2 tipos de operadores pelo menos)	Arthur e Heitor
---	Persistência de Objetos (via arquivo de texto ou binário): AMBOS	
4.3	- Persistência de Objetos.	Arthur e Heitor
4.4	- Persistência de Relacionamento de Objetos.	Não utilizado
5	Virtualidade: AMBOS	
5.1	- Métodos Virtuais Usuais.	Arthur e Heitor
5.2	- Polimorfismo.	Arthur e Heitor
5.3	- Métodos Virtuais Puros / Classes Abstratas.	Arthur e Heitor
5.4	- Coesão/Desacoplamento efetiva e intensa com o apoio de padrões de projeto (mais de 5 padrões).	Não utilizado
6	Organizadores e Estáticos: AMBOS	
6.1	- Espaço de Nomes (<i>Namespace</i>) criada pelos autores.	Arthur e Heitor
6.2	- Classes aninhadas (<i>Nested</i>) criada pelos autores.	Arthur e Heitor
6.3	- Atributos estáticos e métodos estáticos.	Arthur e Heitor
6.4	- Uso extensivo de constante (<i>const</i>) parâmetro, retorno, método...	Arthur e Heitor
7	Standard Template Library (STL) e String OO: ...	
7.1	- A classe Pré-definida <i>String</i> ou equivalente. & - <i>Vector</i> e/ou <i>List</i> da STL (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Arthur e Heitor
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Arthur e Heitor
---	Programação concorrente: NÃO REALIZADO	
7.3	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não utilizado
7.4	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não utilizado
8	Biblioteca Gráfica / Visual: AMBOS	
8.1	- Funcionalidades Elementares. & - Funcionalidades Avançadas como: tratamento de colisões e duplo <i>buffer</i>	Arthur e Heitor
8.2	- Programação orientada e evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto <i>Observer</i>) em algum ambiente gráfico. OU - RAD – <i>Rapid Application Development</i> (Objetos gráficos como formulários, botões etc).	Não utilizado
---	Interdisciplinaridades via uso de Conceitos de Matemática Contínua e/ou Física: AMBOS.	
8.3	- Ensino Médio Efetivamente.	Arthur e Heitor
8.4	- Ensino Superior Efetivamente.	Não utilizado
9	Engenharia de Software: ...	
9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	Arthur e Heitor
9.2	- Diagrama de Classes em UML.	Arthur e Heitor
9.3	- Uso efetivo e intensivo de padrões de projeto <i>GOF</i> , i.e., + de 5 padrões.	Não utilizado
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Arthur e Heitor
10	Execução de Projeto: AMBOS	
10.1	- Controle de versão de modelos e códigos automatizado (via github). & - <i>Uso de alguma forma de cópia de segurança (i.e., backup).</i>	Arthur e Heitor

10. 2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO A ENTREGA DO TRABALHO]	Arthur e Heitor
10. 3	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA]	Arthur e Heitor
10. 4	- Escrita do trabalho e feitura da apresentação & - Revisão do trabalho escrito de outra equipe e vice-versa.	Arthur e Heitor (Lucas Schaefer e Matheus Valente)

- Arthur trabalhou em 100% das atividades as realizando ou colaborando nelas efetivamente.
- Heitor trabalhou em 100% das atividades as realizando ou colaborando nelas efetivamente.

AGRADECIMENTOS PROFISSIONAIS

Gostaríamos de agradecer aos monitores, pessoal do PETECO e ao professor Jean Marcelo Simão pelo apoio e pela assistência ao longo do desenvolvimento deste trabalho. O conhecimento e as experiências compartilhadas foram essenciais para alcançar o objetivo final.

Também agradecemos à equipe que revisou este trabalho, cujas sugestões e comentários contribuíram significativamente para o aprimoramento do conteúdo e da apresentação final.

REFERÊNCIAS CITADAS NO TEXTO

- [1] SFML. *Versão 2.6.2* [Biblioteca]. Paris: Simple and Fast Multimedia Library, 2025. Disponível em: <https://www.sfml-dev.org/>. Acesso em: 2 fev. 2025.
- [2] GITHUB. *Plataforma de controle de versão e hospedagem de código-fonte* [Software]. São Francisco: Microsoft, 2025. Disponível em: <https://github.com/>. Acesso em: 2 fev. 2025.
- [3] STARUML. *Versão 5.0.2* [Software]. Seul: MKLab, 2025. Disponível em: <https://staruml.io/>. Acesso em: 2 fev. 2025.

REFERÊNCIAS UTILIZADAS NO DESENVOLVIMENTO

- [A] REFACTORING GURU. *Design Patterns*. Disponível em: <https://refactoring.guru/pt-br/design-patterns>. Acesso em: 26 dez. 2024.
- [B] Gege++. *Criando um Jogo em C++ do ZERO*. Youtube, 2024. Disponível em: https://www.youtube.com/playlist?list=PLR17O9xbTbIBBoL3lli44N8LdZVvg-_uZ. Acesso em: 9 dez. 2024.
- [C] CHATGPT. *Assistência ocasional no desenvolvimento do jogo*. OpenAI, 2025. Disponível em: <https://chat.openai.com/>. Acesso em: 2 fev. 2025.
- [D] STARUML. *Versão 5.0.2* [Software]. Seul: MKLab, 2025. Disponível em: <https://staruml.io/>. Acesso em: 2 fev. 2025.

- [E] VISUAL STUDIO CODE. *Versão 1.96.4* [Software]. Redmond: Microsoft, 2025. Disponível em: <https://code.visualstudio.com/>. Acesso em: 2 fev. 2025.
- [F] GITHUB. *Plataforma de controle de versão e hospedagem de código-fonte* [Software]. São Francisco: Microsoft, 2025. Disponível em: <https://github.com/>. Acesso em: 2 fev. 2025.
- [G] SFML. *Versão 2.6.2* [Biblioteca]. Paris: Simple and Fast Multimedia Library, 2025. Disponível em: <https://www.sfml-dev.org/>. Acesso em: 2 fev. 2025.
- [H] *Manual Project Simas*. 2025. Disponível em: <https://docs.google.com/document/d/1zZhXefhxtHiROkwvyBsDSwLcbZ6xUYBJE1wMBnSyOuE/edit?tab=t.0>. Acesso em: 2 fev. 2025.
- [I] GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Design patterns: elements of reusable object-oriented software*. Boston: Addison-Wesley, 1994.
- [J] SIMÃO, J. M. Site das Disciplinas Técnicas de Programação, Curitiba – PR, Brasil. Acesso em: 2 fev. 2025.
<https://pessoal.dainf.ct.utfpr.edu.br/jeansimao/Fundamentos2/Fundamentos2.htm>
- [K] PISKEL. *Piskel - Free online sprite editor*. Versão [informe a versão, se disponível]. Disponível em: <https://www.piskelapp.com/>. Acesso em: 4 fev. 2025.
- [L] YESCHAT.AI. *YesChat: inteligência artificial de conversação*. Disponível em: <https://yeschat.ai>. Acesso em: 23 dez. 2024.
- [M] KELLY, Austin. *Soldado com arma, munição, atirando, Segunda Guerra Mundial, folha de sprites, diferentes ângulos*. 2023. Disponível em: <https://www.seaart.ai/pt/explore/detail/clmp4894msbc739bu0r0>. Acesso em: 10 dez. 2024.
- [N] LOVEPIK. *Q Version Cartoon Horizontal Version Of The Game Character Scene*. 2023. Disponível em: <https://lovepik.com/image-610487496/q-version-cartoon-horizontal-version-of-the-game-character-scene.html>. Acesso em: 16 dez. 2024.
- [O] FOXYFOX123. *Pixel mar panorama. 8 bits Sol de praia com aceno, nuvem e areia. Jogos verão oceano panorama. Nublado azul céu com horizonte fundo. Píxeis ilha cena. Vetor ilustração*. 2023. Disponível em: <https://pt.vecteezy.com/arte-vetorial/37047308-pixel-mar-panorama-8-bits-sol-de-praia-com-aceno-nuvem-e-areia-jogos-verao-oceano-panorama-nublado-azul-ceu-com-horizonte-fundo-pixeis-ilha-cena-vetor-ilustracao>. Acesso em: 26 dez. 2024.
- [P] DJVSTOCK. *Pixelated red flag*. 2023. Disponível em: <https://www.vectorstock.com/royalty-free-vector/pixelated-red-flag-vector-41549601>. Acesso em: 4 fev. 2025.

[Q] **SHUTTERSTOCK.** Black hole pixel art flat icon. 2023. Disponível em: <https://www.shutterstock.com/pt/image-vector/black-hole-pixel-art-flat-icon-2201194677>. Acesso em: 2 fev. 2025.